

大数分解 筛法, 二次筛法
数域筛法

②

双筛记(下)

(纪念我的朋友和老师 Paul Erdős)

Carl Pomerance*

Pomer., C

实现与改善

冯克勤 0156

对我来说很幸运的是, 二次筛法真的成为有竞争能力的算法. 通常人们只用复杂性推理和思想上的实验发明了某种算法, 后来常常会发现是没有竞争能力的方法. 此外, 即使基本思想是对的, 也需要等待实际使用的人们加以改善. 对于二次筛法就经历了这样的过程.

用二次筛法作大数分解的第一个人是 Joseph Gerver(见 [9]). 他在 Cunningham 课题中利用学习程序的机会, 成功地分解了一个 47 位数, 这个课题是由 Allan J. Cunningham 和 H.J. Woodall 于本世纪早期开始的, 任务是分解所有数 $b^n \pm 1$ 为素因子乘积, 其中 b 从 1 到 12(但不是某个数的幂), 而 n 一直跑到很大的数(见 [3]). Gerver 分解的数是 $3^{225} - 1$.

当时我找人试验二次筛法是相当不容易的, Cunningham 课题中许多数用连分数方法分解似乎都是相当满意的, 所以他们认为, 与连分数方法中的数 Q_i 相比 Kraitchik 多项 $Q(x)$ 数值太大, 这对于乳臭未干的二次筛法是一个很大缺欠. 但是在 1982 年秋天的 Winnipeg 会议上, 我说服 Sandia 实验室的 Gus Simmons 和 Tony Warnock 在他们的 Cray 计算机上试一下二次筛法.

Jim Davis 和 Diane Holdridge 受命把二次筛法编制到 Sandia Cray 机器上. 他们不仅成功地完成了这项任务, 而且很快便创造记录. Davis 发现了一个重要改进, 从而消除了上面提到的缺欠. 当 $Q(x) = x^2 - n$ 的数值逐渐增大时, 他找到用其他二次多项式的一种办法. 这种思想并没有使复杂性估计有本质变化, 但却使方法更加实用. 他们的成功不仅上了 Mathematical Intelligencer 的封面(1984 年第 6 卷第 3 期封面登有 Cary 计算机和由 71 个 1 组成的数的分解式), 而且《时代》杂志也登载了一篇短文和 Simmons 的照片.

具有讽刺意义的是, 就在 Sandia 这个小组即将开始工作的时候, 另一个 Sandia 小

原题: A Tale of Two Sieves. 译自: Notices of the American Mathematical Society, Vol. 43, No. 12, 1996, pp. 1473-1485.

*Carl Pomerance 是美国 Georgia 大学数学系教授, 计算数论学家, 大数分解的二次筛法创始人. ——译注.

组设计和制作了用于公开密钥体制的一个 RSA 版块, 而这个版块的安全性是建立在我们不能分解 100 位数字的基础之上的. 这显然是不够安全的, 所以这个版块变成废物.

差不多与此同时 Peter Montgomery 也独立地发明了另一个更好些的方式来改变多项式. 我们现在使用他的方法而不是 Davis 方法.

与连分数方法相比, 二次筛法有一个很大的优点, 就是特别容易把分解任务同时分配给许多计算机. 例如在使用许多多项式时, 每个计算机在作筛法时都可用给它自己的二次多项式集合. 二次筛法的巨大成功首先得自超级计算机, 像 Sandia 实验室的 Cray XMP. 随着价格便宜的工作站和 PC 机的普遍采用以及二次筛法可以分配工作的自然方式, 新的记录逐渐转移到对目标数组织分配攻击方案的工作中来.

在多个机器上实现因子分解的第一个人是 Robert Silverman. 后来 Red Alford 和我用 100 个最原始的不连网 PC 机分解了一些 100 位的数 (见 [2]). 但是我们没有创造记录, 因为我们的方法太笨. Arjan Lenstra 和 Mark Manasse[12] 把分配工作进行得更加彻底. 他们把二次筛法放到 Internet 上, 把计算机时间敬送给全世界的人们, 经过这样共同分担的努力在 1994 年分解了 129 位的 RSA 挑战数. 由 Derek Atkins, Michael Graff, Paul Leyland 和 Lenstra 领导的这项工作共实时八个月, 包含 10^{17} 个基本步骤.

二次筛法是一个非常简单的算法, 这也是它的威力所在. 由于它非常简单, 就会想到设计一个专用机专门来作大数分解. Georgia 大学的 Jeff Smith 和 Sam Wagstaff 已经建立了专门机器实现连分数算法. 这个浑名为“乔治亚破碎机”的机器取得了有限的成功, 又被在通常机器上能进行二次筛法分解投上了阴影. Smith, Randy Tuler 和我 (见 [21]) 想到, 我们也许可以建造二次筛法的专用机. 我们建造了这样的专用机 “Quasimodo”, 但是没有发挥功能. 由于低价高质量计算机的迅速普及, 我们的想法后来成了问题.

数域筛法的诞生

受到 Don Coppersmith, Andrew Odlyzko 和 Richard Schroepel [6] 利用二次数域构造离散对数算法的启发, John Pollard 在 1988 年写了一封信寄给一些人传看, 信中描述了他利用代数数域分解某些大数的想法. 他的原始思想并不是用来分解任意的大合数, 而只用于分解某些“好”的合数, 即它们接近于方幂并且还具有另一些好的性质. 他用分解第 7 个费马数 $2^{2^7} + 1$ 来说明他的想法. 有趣的是, 大约在二十年前连分数方法所取得的第一个重大成功就是分解这个数.

我必须承认, 一开始我没有太理会 Pollard 的方法, 因为它似乎只适合用于分解很少的数. 但是有些人对这个方法作了认真的考虑, 其中一个人就是 Hendrik Lenstra. 他改进了算法的某些细节, 并且与 Mark Manasse 和他的兄弟 Arjen 在 Cunningham 课题中用这种方法分解了一些大数 (最引人注意的是一个 138 位数). 在 Brian LaMacchia 和 Andrew Odlyzko 对于此方法中出现的大规模稀疏矩阵给出好的处理方式之后, Lenstra 兄弟和 Manasse 把眼睛盯在第 9 个费马数 $2^{2^9} + 1$ 上 (这个数在 Pollard 原始信件中就被建议为是值得考虑的目标, 已经知道它是合数, 即已知它的一个 7 位数的素因子, 而

且也知道剩下的 148 位数因子仍是合数, 但不知它的分解). 这显然是二次筛法不能胜任的. Hendrik Lenstra 自己在 1985 年发明的椭圆曲线方法, 它特别适合用于分解具有比较小素因子的数 (例如素因子不超过 30 位), 那时也不能分解它. 在 1990 年春天 Lenstra 弟兄二人和 Manasse 成功地分解了 $2^{2^9} + 1$. 这项进展向世界宣布 Pollard 的数域筛法时代已经到来.

但是对一般的数进行分解情况会怎样? 1989 年我在加拿大数论协会渥太华会议上有个演讲, 对因子分解作了综述, 提到 Pollard 的新方法. 在开会的飞机上我对这种方法作了复杂性分析, 考查它对分解一般的数是否有效 (假定不存在太多的技术困难并且假定这种方法对一般的数行). 结果使我很惊奇, 这个 (还不存在的) 算法的复杂度为 $\exp(c(\log n)^{1/3}(\log \log n)^{2/3})$. 它与二次筛法复杂度的关键区别是在指数函数中 $\log n$ 的方幂由 $1/2$ 降成 $1/3$. 试想从连分数方法转成二次筛法的巨大动力不过是把指数函数中的常数加以改进, 那末降低了指数函数中的指数将会发生怎样的情况! 显然这种方法很值得认真考虑.

我不想给人印象是我独自用复杂性分析找到了一种方式把数域筛法用于一般的合数. 一点也不. 我只是隐约地看到了令人激动的未来可能性. 这些可能性主要是由 Joe Buhler, Hendrik Lenstra 和其他人实现的. 此外, 在几个月之前 Lenstra 对于 Pollard 方法用于一些特殊数也作了复杂性分析, 他也得到表达式 $\exp(c(\log n)^{1/3}(\log \log n)^{2/3})$. 我自己的分析是对一般的数作的, 但是基于某些乐观的代数假定和期望能够保持的平均性假定.

对 n 作因子分解的 Pollard 方法是从一个整系数首 1 不可约多项式 $f(x)$ 和一个整数 m 开始, 其中 $f(m) \equiv 0 \pmod{n}$. 这个多项式具有“适度”的次数 d . 其意思指的是: 若 n 为 100 至 200 位的数, 则 d 应当为 5 或 6. 例如对第 9 个费马数 $n = 2^{2^9} + 1$, 容易得到这样的多项式: 注意 $8n = 2^{515} + 8$, 所以可取 $f(x) = x^5 + 8$ 和 $m = 2^{103}$.

这样一个多项式有什么用处? 设 α 是 $f(x)$ 的一个复数根, 并考虑环 $\mathbb{Z}[\alpha]$. 由于 $f(\alpha) = 0$ 和 $f(m) \equiv 0 \pmod{n}$, 将 α 映成同余类 $m \pmod{n}$ 便得到从 $\mathbb{Z}[\alpha]$ 到 $\mathbb{Z}/n\mathbb{Z}$ 的一个自然映射 ϕ . 我们关于 f, α 和 m 的条件保证 ϕ 是可定义的, 并且 ϕ 是环同态.

现在设 S 是具有下述两条性质的一些互素整数对 $\langle a, b \rangle$ 构成的有限集合. 第一个性质是: $\prod_{\langle a, b \rangle \in S} (a - \alpha b)$ 是 $\mathbb{Z}[\alpha]$ 中平方数 γ^2 . 第二个性质是: $\prod_{\langle a, b \rangle \in S} (a - mb)$ 是 $\mathbb{Z}$ 中平方数 v^2 . 由于 γ 可表成 α 的整系数多项式, 从而将 α 代之以整数 m , 则有整数 u 使得 $\phi(\gamma) \equiv u \pmod{n}$. 于是

$$\begin{aligned} u^2 &\equiv \phi(\gamma)^2 = \phi(\gamma^2) = \phi\left(\prod_{\langle a, b \rangle \in S} (a - \alpha b)\right) \\ &= \prod_{\langle a, b \rangle \in S} \phi(a - \alpha b) \equiv \prod_{\langle a, b \rangle \in S} (a - mb) = v^2 \pmod{n} \end{aligned}$$

我们已经知道 u 和 v 有什么用途. 就象 Kraitchik 在七十年前告诉我们的那样, 我们希望 $u \not\equiv \pm v \pmod{n}$. 这时求最大公因子 $(u - v, n)$ 就给出 n 的一个非平凡因子.

怎样才能得到满足条件的 $\langle a, b \rangle$ 所构成的集合? 至少对于第二个性质 (即所有数

$a - mb$ 的乘积是平方数). 我们显然仍可用指数向量和筛法, 这时有两个变量 a 和 b 而不象二次筛法中的 $Q(x)$ 只有一个变量. 从而我们可把它看成为参数化了的线性多项式族. 我们可以固定 b 而令 a 跑过一个区间, 然后改成下一个 b 再重复下去.

但是 S 也必须具有第一个性质, 即对同样一些数对 $\langle a, b \rangle$, 所有 $a - ab$ 的乘积是 $\mathbb{Z}[\alpha]$ 中平方. Pollard 想到: 如果我们处在最理想的情形, 即 $\mathbb{Z}[\alpha]$ 是 $Q(\alpha)$ 的整数环并且是唯一因子分解整环, 而我们又知道单位群的一组基, 则可以对代数整数 $a - ab$ 同样建立指数向量, 然后基本上重复同样的算法, 为了使 S 的上述两条性质同时成立, 不过是利用长一些的指数向量, 它的分量分别对应于所有小的素数, $a - ab$ 的符号, $\mathbb{Z}[\alpha]$ 中所有“小”的素数以及单位基中的每个单位.

但是如何对一般的数 n 作这些事情? 作为第一步我们如何求得多项式 $f(x)$ 和整数 m 使得 $f(m) \equiv 0 \pmod{n}$? 如果能找到 $f(x)$ 和 m , 为什么我们希望 $\mathbb{Z}[\alpha]$ 有上述好的性质才能使 Pollard 的方案能够工作?

再读数域筛法

至少在开始求 $f(x)$ 和 m 时我们有简单的办法. 我们把决定 $f(x)$ 放在最后, 而首先定下来 f 的次数 d . 然后取 m 为 $n^{1/d}$ 的整数部分. 再把 n 作 m 进制展开: $n = m^d + c_{d-1}m^{d-1} + \cdots + c_0$ 其中数字 c_i 满足 $0 \leq c_i < m$ (如果 $n > (2d)^d$, 则首位 c_d 必然为 1.) 现在取 $f(x)$ 为 $x^d + c_{d-1}x^{d-1} + \cdots + c_0$ 即可. 它是整系数首 1 多项式, 但它是不可约吗?

现在已有许多好的方法把 $\mathbb{Z}$ 上一个本原多项式分解成不可约多项式的乘积¹⁾. 事实上, Lenstra 兄弟和 László Lovász 给出了分解 $\mathbb{Z}[x]$ 中本原多项式的一个漂亮的多项式时间算法 (所需时间以多项式次数与各系数的位数之和的某个方幂为上界). 假如我们非常不走运, $f(x)$ 是可约多项式并且用上述算法得到 $f(x) = g(x)h(x)$, 则 $n = f(m) = g(m)h(m)$. 由 John Brillhart, Michael Filaseta 和 Andrew Odlyzko 的一个结果可知 n 的这个分解是不平凡的. 而我们的目的正是求 n 的非平凡分解, 所以这不是不走运! 由于几乎所有多项式都是不可约的, 所以对于上面构造的 $f(x)$ 多数情况是要进行数域筛法, 即我们不能马上得到 n 的分解.

还有一个主要问题是: 环 $\mathbb{Z}[\alpha]$ 不一定具有上述那些好的性质, 这个困难如何克服? 1990 年前后, Joe Buhler, Hendrik Lenstra 和我共同克服了剩下来的困难. Len Adleman [1] 又加进一个很实际的想法, 它简化了我们的一些处理方式, 在 [11] 中发表了所谓一般数域筛法.

现在概括介绍一下我们作了些什么, 易知 $N(a - ab)(a - ab \text{ 对于 } \mathbb{Q} \text{ 的范数})$ 等于 $b^d f(a/b)$, 这是 f 的齐次形式. 我们定义 $a - ab$ 是 Y -光滑的, 是指 $N(a - ab)$ 是 Y -光滑的. 由于范是积性函数, 所以若一些代数整数 $a - ab$ 的乘积为某个代数整数的平方, 则对应范的乘积是一个整数的平方, 而我们知道如何求得一些 $\langle a, b \rangle$ 使诸 $N(a - ab)$

¹⁾ 本原是指多项式诸系数的最大公因子为 1.——译注.

的乘积为平方数,即可以用筛法找到 $N(a - \alpha b)$ 的 Y -光滑值,然后由 $\mathbb{F}_2$ 上的指数向量将它们作适当的组合.

一些 $N(a - \alpha b)$ 之积为平方数,只是对应 $a - \alpha b$ 之乘积为 $\mathbb{Z}[\alpha]$ 中平方数的一个必要条件,远不是充分条件,它的根本原因是:范映射可以把不同素理想映成同一个整数,所以 $N(\beta)$ 比 β 更容易成为平方.例如在 $\mathbb{Z}[i]$ 中 $2+i$ 和 $2-i$ 的范均是 5. $(2+i)(2-i) = 5$ 的范为 5^2 ,但是 $(2+i)(2-i) = 5$ 不是 $\mathbb{Z}[i]$ 中平方.(注意:如果在 $\mathbb{Q}(\alpha)$ 的整数环上工作,则对每对互素的整数 a 和 b , $a - \alpha b$ 的素理想因子都是一次的,即它们的范都是有理素数.)对每个素数 p ,以 R_p 表示 $f(x) \equiv 0 \pmod{p}$ 的所有解构成的集合.如果我们碰到 $\langle a, b \rangle$ 使得 p 除尽 $N(a - \alpha b)$,则 p 上的某个素理想除尽 $a - \alpha b$.我们可以判定这是哪个素理想,因为 a/b 模 p 同余于 R_p 中的某个成员,所以它可以用来分清 p 上的所有素理想.于是我们可以安排我们的指数向量,对每个 p 都有 $\#R_p$ 个坐标分量,这就得到了 $a - \alpha b$ 的素理想分解.注意 $\#R_p$ 不超过 $f(x)$ 的次数 d .

这样我们就越过了主要障碍,但仍然还有许多障碍.我们是工作在环 $\mathbb{Z}[\alpha]$ 上,它不一定是 $\mathbb{Q}(\alpha)$ 的整个整数环.事实上, $\mathbb{Z}[\alpha]$ 都不一定为 Dedekind 环,所以每个元素都不一定有素理想因子分解.而且即使分解成素理想之积,当一些代数整数 $a - \alpha b$ 之积是某个理想的平方时,它也不一定是主理想的平方.即使它是主理想的平方,它也不一定是某个元素的平方,因为还有单位问题.(例如由 -9 生成的主理想是 $\mathbb{Z}$ 中理想的平方,但 -9 不是平方数.)即使一些 $a - \alpha b$ 之积是一个代数整数的平方,如何知道它是 $\mathbb{Z}[\alpha]$ 中某个元素的平方?

最后一个障碍比较容易处理,只要加 $f'(\alpha)^2$ 作为因子(定理:若 f 是 $\mathbb{Z}$ 上首 1 不可约多项式, α 为 f 的一个复数根.如果 r 是 $\mathbb{Q}(\alpha)$ 中代数整数,则 $f'(\alpha)r \in \mathbb{Z}[\alpha]$. 因此若 r^2 是 $\mathbb{Q}[\alpha]$ 的整数环中的平方,则 $f'(\alpha)^2 r^2$ 是 $\mathbb{Z}[\alpha]$ 中的平方)其他障碍看起来都比较难以克服.但是 Len Adleman [1] 的一个简单而聪明的想法可以一下子越过所有的障碍.关键之处在于,即使我们面临一些难以对付的障碍,它们模平方数之后只构成 $\mathbb{F}_2$ 上一个相当小维数的向量空间.所以一开始曾经觉得可以完全忽略这些问题,但是维数还没有小到这样程度. Adleman 建议随机地选取某些二次特征,用它们在 $a - \alpha b$ 处的值来增大指数向量.(但在算法开始时要选定二次特征.)然后我们安排一些 $a - \alpha b$,使它们的乘积不仅与一个平方相差“障碍空间”的元素,而且也非常象是一个平方.例如上面问题所考虑的 -9 不是平方.如果某种原因我们看不出符号上的问题,那么它就象是一个平方,因为对每个素数 p , -9 的因子分解中 p 的指数都是偶数,所以我们仍须检查这个问题.作法是考虑某个二次特征在 -9 处的取值 $(-9/p)$,这量 $(-9/p)$ 是勒让得符号.当 -9 为模 p 平方时 $(-9/p) = 1$,否则 $(-9/p) = -1$.假如我们用 $p = 7$ 来试验,容易计算出 $(-9/7) = -1$,所以 -9 不是模 7 的平方,从而也不是 $\mathbb{Z}$ 中平方.当然,若对某个 p , -9 是模 p 的平方也不能保证它是 $\mathbb{Z}$ 中平方.例如取 $p = 5$,则 -9 仍然象是一个平方. Adleman 的想法是计算 $a - \alpha b$ 在选定的一些二次特征上的值,然后用线性代数造出具有如下两个性质的元素:它的(未被增大的)指数向量所有坐标均为偶数,并且在每个选定二次特征上的取值也均为 1.这个代数整数在直觉上就象是平方.如果它不是平方,

可以继续用 $\mathbb{F}_2$ 上线性代数造出另一个候选者.

当然还有困难. 其中一个困难就是“求平方根问题”. 如果我们知道一些有理整数的因子分解式并且它们的乘积是平方数, 用这些因子分解式很容易算出乘积的平方根. 但在 $\mathbb{Z}[\alpha]$ 中这个问题看起来并不这么清楚. 不论如何现在已经有了解决这个问题的办法, 尽管在算法中仍存在着减少这一步骤的计算量问题. 有兴趣的读者可参阅 [15].

也许大家不清楚为什么数域筛法是一个好的因子分解算法. 在二次筛法或者数域筛法这类因子分解方法中, 一个关键的量是我前面表示的“ X ”. 它是对我们所希望组合成平方的那些辅助数有多大的一个估计量. X 所造成的复杂度大约为 $\exp(2\sqrt{\log X \log \log X})$. 在二次筛法中 X 大约为 $n^{1/2+\varepsilon}$. 而在数域筛法中可以选取多项式 $f(x)$ 和整数 m , 使得 (我们希望的光滑) 数 $(a - mb)N(a - ab)$ 有上界 $X = \exp(c'(\log n)^{2/3}(\log \log n)^{1/3})$. 所以为了筛出光滑者我们所用的辅助数的位数差不多是 $(n \text{ 的位数})^{2/3}$, 而二次筛法中辅助数的位数超过 n 的位数的一半, 这就是为什么在渐近意义下数域筛法速度较快.

我在前面已经说过, 用数域筛法分解 n 所需时间大约为 $X = \exp(c'(\log n)^{1/3}(\log \log n)^{2/3})$, 但我还没有解释 c 是什么数. 实际上, 根据所用数域筛法不同方案, c 可以有三个不同数值. 一种很象 Pollard 原始方法的“特殊”数域筛法, 有 $c = (32/9)^{1/3} \approx 1.523$. 这种方法特别适用接近于高次幂的整数 (诸如 $2^{2^9} + 1$). 而对我在本文中介绍的“一般”数域筛法则 $c = (64/9)^{1/3} \approx 1.923$, 它可以分解任何奇合数. 最后, Don Coppersmith [5] 建议了一般数域筛法的另一个方案, $c = \frac{1}{3}(92 + 26\sqrt{13})^{1/3} \approx 1.902$, 这种筛法可以采用许多多项式, 但似乎是渐近最差的分解方法. 所以 Coppersmith 的想法曾被认为是完全不实际的, 但是在 [8] 中考虑到这种方法使用多个多项式, 可能有某种实际的好处.

筛法艺术的现状

1996 年 4 月, 一个大的工作队伍 (见 [7]) 用一般的数域筛法分解了 130 位的 RSA 挑战数, 从而二次筛法终于交出了金腰带, 而由于分解了最大的“硬”数, 二次筛法从 1983 年起曾经一直处于冠军地位. 尽管在两年前用数域筛法分解 129 位挑战数时所用的时间与二次筛法差不多一样, 但是人们估计数域筛法所用的计算时间只占 15%. 这个差别是由于工作中使用的机器较少, 并且在机器上编制算法最后阶段时浪费了一些时间.

那么, 二次筛法与数域筛法的交叉点在哪里? 这个问题的答案是因人而异的. 但有一点是所有人都同意的: 对于比较小的数 (例如小于 100 位的数) 用二次筛法比较好, 而对较大的数 (例如超过 130 位的数) 则用数域筛法比较好. 这类问题没有简单答案, 是因为它与所编程序的细节和使用机器的类型有很大关系. 例如在 [7] 中说, 数域筛法的好坏要看计算机有多大的存贮量. 二次筛法也是如此, 但是对存贮量的依赖程度没有那么大.

在这篇综述文章中还有许多事情没有谈及. 一个重要的没有涉及的问题是二次筛法和数域筛法线性代数部分的算法和复杂性的讨论. 开始我们使用过高斯消去法, Brillhart 和 Morrison 的连分数方法就是用了这种方法. 但是矩阵的大小在不断增加. 现

在对于创纪录的因子分解, 因子基中有一百万个素数已是习以为常, 所以处理一百万阶方阵的线性代数问题变得相当重要, 这方面已经有不少有趣的新工作, 把处理实数域上稀疏矩阵的迭代方法移植到二元域 F_2 上, 近来的有关文献可见 [14].

将数域筛法的基本思想加以某些变化会有好处. 一种变化是把数域筛法中使用的线性表达式 $a - mb$ 改成 $b^k g(a/b)$, 其中 $g(x)$ 为 $\mathbb{Z}[x]$ 中 k 次不可约多项式, 并且 $g(m) \equiv 0 \pmod{n}$. 也就是说, 我们使用两个多项式 $f(x)$ 和 $g(x)$, 它们有模 n 的公共根 m (开始时我们曾取 $g(x) = x - m$). 如何选取这两个多项式是当前一个重要研究课题. 对通常数域筛法的另一种变化是按照 Coppersmith 采取的路线将多项式 $f(x)$ 改成一组多项式, 在 [8] 中描述了如何在数域筛法中把上述两种变化组合在一起.

离散对数问题也是密码学的一个有趣的问题. 这个问题是说: 给定由 g 生成的一个循环群, 对此群中每个元素 h 如何求得整数 x 使得 $g^x = h$. 我们说过, Pollard 关于数域筛法的原始思想就源于一个离散对数算法. Dan Gordon, Oliver Schirokauer 和 Len Adleman 又使用数域筛法的各种变型来计算有限域乘法群的离散对数, 所以我们走了一个圆圈. 关于这方面近来发展的综述可参见 [22].

我还没有谈到素数判定这个题目, 一般来说, 判别一个数是合数比将它完全分解要容易得多. 在我们用复杂而耗时的算法分解一个数的时候, 我们总是先用其他方法来试验一下它为奇的合数并且不是某个数的幂.

我只是偶然提到过 H. Lenstra 的大数分解椭圆曲线方法, 除了合数的一个稀疏集合之外, 这个算法比二次筛法和数域筛法都优越得多, 这个稀疏集合就是所谓“硬”数集合. 对“硬”数我们仍采用筛法.

因子分解课题也有一个严格化的方面, 即研究人员试图补偿启发式推理的不足而对于因子分解算法证明某些定理. 到目前为止, 与确定性算法相比较, 在概率性算法方面证明定理已取得较大的成功. 对于象二次筛法和数域筛法这些已实际应用的各种筛法, 我们似乎还没有接近于可以证明它们工作起来能达到所宣传的有效程度. 所幸的是, 在我们试图分解各种数时还没有发现这种缺乏证明给工作带来什么问题.

关于进一步阅读材料, 我建议前面已经提到的文献以及 [10, 13, 17, 18, 19, 20]. 此外, 我目前正在与 Richard Crandall 写一本书《Primes: A computational perspective》, 这本书期望在 1997 年出版.

我希望我已经向读者介绍了二次筛法和数域筛法近来发展的某些思想和令人激动的情景. 从这些发展可以看到复杂性理论估计与好算法的直观探索之间的相互影响. 缺乏任何一方面, 都不可能由另一方面把我们带到目前的进展位置上.

前 20 个费马数

m	$F_m = 2^{2^m} + 1$ 的分解
0	3
1	5
2	17
3	257
4	65537

- 5 641 · P_7
- 6 274177 · P_{14}
- 7 59649589127497217 · P_{22}
- 8 1238926361552897 · P_{62}
- 9 2424833·7455602825647884208337395736200454918783366342657 · P_{99}
- 10 45592577·6487031809·4659775785220018543264560743076778192897 · P_{252}
- 11 319489·974849·167988556341760475137·3560841906445833920513 · P_{564}
- 12 114689·26017793·63766529·190274191361·1256132134125569 · C_{1187}
- 13 2710954639361·2663848877152141313·3603109844542291969·
319546020820551643220672513 · C_{2391}
- 14 C_{4933}
- 15 1214251009·2327042503868417 · C_{9840}
- 16 825753601 · C_{19720}
- 17 31065037602817 · C_{39444}
- 18 13631489 · C_{78906}
- 19 70525124609·646730219521 · C_{157804}

表中 P_k 表示一个 k 位的素数, C_k 表示 k 位的合数 (但是还不知它如何分解).

费马数的分解历史是整个因子分解历史的一个缩影. 费马自己知道从 F_0 到 F_4 都是素数, 然后他猜想所有形如 $2^{2^m} + 1$ 的数都是素数. 但是欧拉发现了 F_5 的分解. 这个分解是不难发现的, 因为可以用本质上由费马得到的如下的结果: 当 $m \geq 2$ 时, 如果 p 是 F_m 的素因子, 则必然 $p \equiv 1 \pmod{2^{m+2}}$. 所以 F_5 的素因子都模 128 同余于 1. 第一个这样的素数 (并且不是费马数) 为 641. 用同样的思想 Landry 于 1880 年得到 F_6 的分解, 后来又有人发现许多其他费马数的“小”素因子, 其中有 80 多个未能在上面表中列出.

F_7 的分解是 Brillhart-Morrison 连分数因子分解方法的首次战绩, 而 Brent 和 Pollard 用 Pollard 的“ ρ ”方法的一种变型分解了 F_8 . 正如正文中所说, F_9 是用数域筛法分解的. F_{10} 和 F_{11} 是 Brent 用 Lenstra 的椭圆曲线算法分解的.

我们知道 F_{14} , F_{20} 和 F_{22} 是合数, 但是不知它们的素因子. 可以用下面的 Pepin 判别法证出它们是合数: F_m 为素数的充分必要条件为 $3^{\frac{F_m-1}{2}} \equiv -1 \pmod{F_m}$. F_{24} 是最小的费马数, 我们还不知道它是素数还是合数. 现在许多数论学家认为在 F_4 之后的所有费马数都是合数.

费马数与欧几里德的一个古典问题有关, 这个问题是: 对哪些 n 可以用圆规直尺构造正 n 边形? 高斯证明了: 正 n 边形是尺规可构造的当且仅当 $n \geq 3$ 并且 n 的最大奇因子是不同的费马素数的乘积. 高斯在 19 岁发现的这个定理伴随他整个一生: 正十七边形被刻在他的墓碑上. (全文续完)

(参考文献略)

(冯克勤译 戴宗铎校)